
ARCHITECTURE BLUEPRINT · MULTI-STORE / MULTI-
LANGUAGE

Necoyoad

v5

*How store_id and language_id compose, the per-store
settings model, the polymorphic object spine, and
widget composition by object_type*

scope: store detection, settings loading, language/currency
detection, object_to_store, object_to_category,
description, property, url_alias, widget composition
findings: no default+override merge, inconsistent o2s writes,
missing store_id in posts insert, dual table usage
listings: 14 PHP · diagrams: 6 · pages: 20

Reverse-Engineered Architecture Report

Verified against PHP source commit ce75adf
github.com/yosietserga/necoyoad

DESCRIPTIVE, NOT PRESCRIPTIVE · NO IMPROVEMENTS PROPOSED

Necoyoad — Multi-Store / Multi-Language Architecture (v5)

How `store_id` and `language_id` compose, the per-store settings model, the polymorphic object spine, and widget composition by `object_type`

Abstract

This is the fifth volume of the Necoyoad architecture blueprint. v5 zooms in on the multi-store and multi-language architecture: how the platform determines which store and which language a request belongs to, how those two dimensions compose to form a unique content surface, and how the polymorphic object spine (`object_to_store`, `object_to_category`, `description`, `property`, `url_alias`) ties content to stores and languages. It also covers the widget composition mechanism that uses the session's `object_type` and `object_id` to render per-page widget trees.

The source reveals several findings that correct or refine v1's inferences: (i) there is *no* “default + override” settings merge — each store loads only its own `store_id` settings; (ii) the `object_to_store` writes are *inconsistent* — products, categories, and manufacturers use legacy column names (`product_id`, `category_id`, `manufacturer_id`) instead of the polymorphic pair (`object_id`, `object_type`); (iii) the posts and post-categories INSERT statements *forget `store_id`*; and (iv) the storefront product model uses *both* `object_to_store` (polymorphic) and `product_to_store` (legacy OpenCart) in different queries. v5 documents all of this, with 14 PHP code listings, 6 TikZ diagrams, and a full multi-store request lifecycle trace.

The stance is unchanged: **descriptive, not prescriptive**. No improvements are proposed.

Keywords: Necoyoad, OpenCart lineage, PHP 8.1.2, multi-store, multi-language, `store_id`, `language_id`, `object_to_store`, `object_to_category`, `description`, `property`, `url_alias`, polymorphic associations, EAV, per-store settings, widget composition, `object_type`, `object_id`, `widget_landing_page`.

Contents

1	Introduction and Scope	4
1.1	Five Volumes	4
1.2	Why Multi-Store / Multi-Language?	4
1.3	Document Roadmap	4
2	Thirty-Second Overview	5
3	Store Detection and STORE_ID Propagation	5
3.1	The Three Detection Strategies	5
3.2	STORE_ID as a Global Constant	6
3.3	The Admin Always Uses STORE_ID = 0	6
4	The Per-Store Settings Model	7
4.1	No Default + Override Merge	7
4.2	The Settings Cache	7
4.3	The setting Table Structure	7
5	Language Detection and Propagation	8
5.1	The 6-Level Detection Priority	8
5.2	Admin Language Detection	9
5.3	language_id Propagation	9
6	Currency Detection	9
7	The Polymorphic Object Spine	9
7.1	The Five Polymorphic Tables	9
7.2	The object_to_store Writes — Inconsistent	10
7.3	The object_to_store Reads — Polymorphic	10
7.4	The description Table — Polymorphic + Language-Keyed	11
7.5	The property EAV Table — Polymorphic + Grouped	11
7.6	The url_alias Table — Polymorphic + Language-Keyed	11
8	Storefront Filtering by store_id and language_id	12
8.1	The Product Model	12
8.2	The Category Model	12
8.3	The Manufacturer Model	13
9	Widget Composition by object_type and object_id	13
9.1	The Session object_type / object_id Protocol	13
9.2	Controller::loadWidgets() Reads the Session	13
9.3	NecoWidget::getRows() Filters by object_type	14
9.4	The widget_landing_page Table	14
10	The $S \times L$ Composition Matrix	15
10.1	How the Matrix is Realised	15
11	Multi-Store Request Lifecycle Trace	16
11.1	Store Detection	16
11.2	Settings Load	16
11.3	Language Detection	16
11.4	SEO URL Resolution	17

11.5 Product Controller	17
12 Cross-Cutting Findings	18
12.1 No Default + Override Settings Merge	18
12.2 Inconsistent <code>object_to_store</code> Writes	18
12.3 Missing <code>store_id</code> in Posts Insert	18
12.4 Dual Table Usage	18
12.5 The Subdomain Regex is Hard-Coded	18
12.6 Language Detection Has No URL-Path Strategy	19
12.7 Currency Override is Per-Request Only	19
12.8 Widget Composition is Two Queries, Not One	19
13 Closing Observations	19
13.1 The Multi-Store Architecture is a Data-Level Multi-Tenancy	19
13.2 The Polymorphic Object Spine is the Architectural Glue	19
13.3 The Inconsistencies are Evolutionary Scars	19
13.4 The Widget Composition Mechanism is the Most Sophisticated Part	19
13.5 Final Note	20
A Glossary	20

1 Introduction and Scope

1.1 Five Volumes

This is the fifth volume. The five volumes operate at different scales:

- **v1** — SQL-only reconstruction (35 pages).
- **v2** — Source verification + mobile widget scoping (38 pages).
- **v3** — Presentation layer / rendering pipeline (40 pages).
- **v4** — Marketing / campaign subsystem (28 pages).
- **v5** — This volume. Multi-store / multi-language architecture.

1.2 Why Multi-Store / Multi-Language?

v1 inferred the multi-store and multi-language patterns from the schema alone. v1 correctly identified the four cross-cutting patterns (polymorphic associations, EAV descriptions, multi-store scoping, multi-language scoping) but could not see the implementation details. v5 verifies v1's inferences against the source and reveals:

1. **No default+override settings merge.** v1 inferred that `store_id=0` is the default and `store_id=N` is the override. The source shows the storefront loads `SELECT * FROM setting WHERE store_id = STORE_ID` — only the current store. There is no merge with `store_id=0`.
2. **Inconsistent object_to_store writes.** The admin store model's `saveContent()` writes products, categories, and manufacturers using legacy column names (`product_id`, `category_id`, `manufacturer_id`) instead of the polymorphic pair (`object_id`, `object_type`). Downloads, pages, posts, and banners use the correct polymorphic pair.
3. **Missing store_id in posts/post-categories inserts.** The `INSERT INTO object_to_store` for posts and post-categories *omits the store_id column* — a bug that means posts and post-categories are not actually scoped to any store.
4. **Dual table usage.** The storefront product model uses *both* `object_to_store` (polymorphic) and `product_to_store` (legacy OpenCart) in different queries. Both tables coexist; some queries hit one, some the other.
5. **6-level language detection.** v1 inferred language detection from URL/cookie/session/browser. The source reveals a 6-level priority: `?language= GET` → `?hl= GET` → `session` → `cookie` → `browser HTTP_ACCEPT_LANGUAGE` → `config_language` setting.
6. **Widget composition by object_type.** v3 covered the widget rendering pipeline but did not explain how the session's `object_type` and `object_id` override the widget tree on a per-page basis. v5 traces this mechanism: product pages set `session->object_type = 'product'`, category pages set `session->object_type = 'category'`, and `Controller::loadWidgets()` passes these to `NecoWidget::getRows()`, which filters with `LIKE '%object_type=product%'` and `LIKE '%object_id=42%'` predicates.

1.3 Document Roadmap

Section 2 gives a 30-second overview. Section 3 covers store detection and `STORE_ID` propagation. Section 4 covers the per-store settings model. Section 5 covers language detection and propagation. Section 6 covers currency detection. Section 7 covers the polymorphic object spine (`object_to_store`, `object_to_category`, `description`, `property`, `url_alias`). Section 8 covers how storefront models filter by `store_id` and `language_id`. Section 9 covers widget composition by `object_type` and `object_id`. Section 10 explains the $S \times L$ composition matrix.

Section 11 is a multi-store request lifecycle trace. Section 12 catalogues the cross-cutting findings.

2 Thirty-Second Overview

A request hits `web/index.php`. The store is detected by one of three strategies (URL path folder match, `?store_id` GET param, subdomain regex). The matching `app/<folder>/config.php` sets the `STORE_ID` constant and `DIR_APPLICATION` path. `app/shop/map.php` loads settings: `SELECT * FROM setting WHERE store_id = STORE_ID` — only the current store, no default merge. The active language is detected via a 6-level priority chain. The active currency is detected from `?cc=` GET param or `config_currency` setting.

When the storefront queries products, it joins `object_to_store` and filters by `p2s.store_id = STORE_ID` and `p2s.object_type = 'product'`. When it queries product descriptions, it joins the polymorphic `description` table and filters by `language_id = config_language_id`. The combination of `store_id` and `language_id` forms a $S \times L$ matrix of content surfaces.

When a product page loads, the product controller sets `session->object_type = 'product'` and `session->object_id = <product_id>`. When `Controller::loadWidgets()` runs, it reads these from the session and passes them to `NecoWidget::getRows()`, which queries the `property` EAV table with `LIKE '%object_type=product%'` and `LIKE '%object_id=42%'` predicates. This returns product-specific widget overrides (e.g. a “related products” widget that only shows on product pages) in addition to the default widgets for the current layout position.

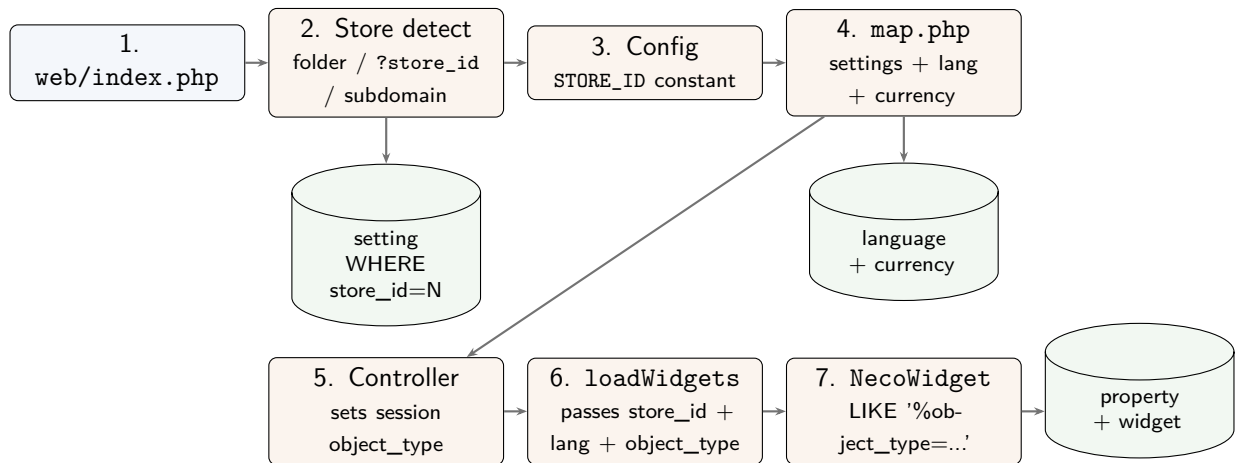


Figure 1: The 7-step multi-store/multi-language pipeline. Store detection → config → settings load (per-store) → language/currency detection → controller sets session `object_type` → `loadWidgets` passes all dimensions to `NecoWidget` → `NecoWidget` queries the `property` EAV table with `LIKE` predicates.

3 Store Detection and STORE_ID Propagation

3.1 The Three Detection Strategies

v2 documented the three store-detection strategies in `web/index.php`. v5 verifies and adds detail:

```

1 require_once($config_path . 'cconfig.php');
2 $db = new mysqli(DB_HOSTNAME, DB_USERNAME, DB_PASSWORD, DB_DATABASE);
3 // Strategy 1: URL path folder match
4 if (isset($_GET['_route_'])) {
5     $parts = explode("/", $_SERVER['REQUEST_URI']);
6     foreach ($parts as $part) {
7         if (empty($part)) continue;
8         $results = $db->query("SELECT * FROM " . DB_PREFIX . "store

```

```

9         WHERE folder = '"' . $db->real_escape_string($part) . '"';
10        $store = $results->fetch_assoc();
11        if ($store['folder']) { $matches[1] = $store['folder']; }
12    }
13    // Strategy 2: ?store_id GET parameter
14    } elseif (isset($_GET['store_id'])) {
15        $results = $db->query("SELECT * FROM " . DB_PREFIX . "store
16        WHERE store_id = '"' . $db->real_escape_string((int)$_GET['store_id'])
17        . '"");
18        $store = $results->fetch_assoc();
19        if ($store['store_id']) { $matches[1] = $store['folder']; }
20    }
21    // Strategy 3: subdomain regex
22    if (!isset($matches[1]))
23        preg_match('/([^.]+\.)\.necoyoad\.com/', $_SERVER['SERVER_NAME'], $matches);
24    if (isset($matches[1]) && $matches[1] != 'www') {
25        if (file_exists($config_path . "app/" . strtolower($matches[1]) .
26            "/config.php")) {
27            require_once($config_path . "app/" . strtolower($matches[1]) .
28                "/config.php");
29        } else {
30            require_once($config_path . 'app/shop/config.php');
31        }
32    } else {
33        require_once($config_path . 'app/shop/config.php');
34    }
35 }

```

Listing 1: Store detection in web/index.php (abridged).

3.2 STORE_ID as a Global Constant

Once the per-app config.php is loaded, STORE_ID is a global PHP constant:

```

1 // app/shop/config.php (default store)
2 define('STORE_ID', 0);
3
4 // app/admin/config.php (admin always operates on the global store)
5 if (!defined("STORE_ID")) define('STORE_ID', 0);
6
7 // app/m/config.php (mobile entry point, hard-coded to store 9)
8 define('STORE_ID', 9);

```

Listing 2: STORE_ID definition in per-app config files.

Because STORE_ID is a `define()` (not a variable), it is available in every scope — every controller, model, library, and template can reference it directly. This is how `store_id` propagates to every SQL query: the storefront models just write `WHERE p2s.store_id = '"' . (int)STORE_ID . '"'` in their SQL.

3.3 The Admin Always Uses STORE_ID = 0

Admin operates on the global store only

The admin entry point (`web/admin/index.php`) loads `app/admin/config.php`, which defines `STORE_ID = 0`. The admin loads settings with `SELECT * FROM setting WHERE store_id = 0`. There is no admin-side store switcher; the admin always manages the global store's settings. Per-store settings for `store_id > 0` are managed through the admin's store-management controller, which writes directly to the `setting` table with the appropriate `store_id`.

4 The Per-Store Settings Model

4.1 No Default + Override Merge

No default+override settings merge

v1 inferred that `store_id=0` is the default and `store_id=N` is the override, with the override merging on top of the default. The source reveals this is *not* the case.

The storefront (`app/shop/map.php` line 48) loads:

```
SELECT * FROM setting WHERE store_id = STORE_ID
```

The admin (`web/admin/index.php` line 32) loads:

```
SELECT * FROM setting WHERE store_id = 0
```

There is no `SELECT ... WHERE store_id IN (0, N)` merge query. Each store must have its own *complete* set of settings. If a setting is missing for `store_id=5`, it is simply not set — there is no fallback to `store_id=0`.

In practice, the admin's store-management controller clones the `store_id=0` settings when creating a new store, so the new store starts with a complete copy. But subsequent changes to `store_id=0` settings do *not* propagate to other stores.

4.2 The Settings Cache

The storefront caches the loaded config in the session:

```
1 if (!$session->has('ntConfig_' . (int) STORE_ID)) {
2     $query = $db->query("SELECT * FROM " . DB_PREFIX . "setting
3     WHERE store_id = '" . (int) STORE_ID . "'");
4     foreach ($query->rows as $setting) {
5         $config->set($setting['key'], $setting['value']);
6     }
7 } else {
8     $config = unserialize($session->get('ntConfig_' . (int) STORE_ID));
9 }
10 $config->set('config_store_id', STORE_ID);
```

Listing 3: Settings caching in `app/shop/map.php`.

So the first request for a store hits the database; subsequent requests in the same session use the cached `Config` object. The cache key includes `STORE_ID`, so switching stores (via a different URL folder or subdomain) correctly loads a different cached config.

4.3 The `setting` Table Structure

Table 1: The `setting` table structure.

Column	Type	Purpose
<code>setting_id</code>	INT	Primary key.
<code>store_id</code>	INT	The store this setting belongs to. 0 = global/admin.
<code>group</code>	VARCHAR(32)	The extension group (e.g. <code>config</code> , <code>mail_server</code> , <code>payment_bank_transfer</code>).
<code>key</code>	VARCHAR(64)	The setting key (e.g. <code>config_title</code> , <code>config_template</code> , <code>mail_server_id</code>).
<code>value</code>	TEXT	The setting value (string; may be serialised PHP for complex values).

The composite (`store_id`, `group`, `key`) is the logical primary key (though the schema has no unique constraint on it). The `value` column stores plain strings for simple settings and serialised PHP for complex ones (e.g. mailserver credentials are stored as `serialize(['server'=>'smtp.gmail.com', 'port'=>'587', ...])`).

5 Language Detection and Propagation

5.1 The 6-Level Detection Priority

The storefront detects the active language via a 6-level priority chain:

```

1 // 1. ?language= GET parameter
2 if (isset($_GET['language']) && array_key_exists($_GET['language'],
    $languages)) {
3     $code = $_GET['language'];
4 // 2. ?hl= GET parameter (also used for cache key)
5 } elseif (isset($_GET['hl']) && array_key_exists($_GET['hl'], $languages)) {
6     $code = $_GET['hl'];
7 // 3. Session
8 } elseif ($session->has('language') &&
    array_key_exists($session->get('language'), $languages)) {
9     $code = $session->get('language');
10 // 4. Cookie
11 } elseif ($request->hasCookie('language') &&
    array_key_exists($request->getCookie('language'), $languages)) {
12     $code = $request->getCookie('language');
13 // 5. Browser HTTP_ACCEPT_LANGUAGE
14 } elseif ($detect) {
15     $code = $detect;
16 // 6. config_language setting (the store's default)
17 } else {
18     $code = $config->get('config_language');
19 }
20
21 // Persist to session and cookie
22 $session->set('language', $code);
23 $request->setCookie('language', $code);
24
25 // Set the language_id in config (used by every SQL query that needs it)
26 $config->set('config_language_id', $languages[$code]['language_id']);
27 $config->set('config_language', $languages[$code]['code']);

```

Listing 4: Language detection in `app/shop/map.php` (abridged).

The browser detection (level 5) parses `HTTP_ACCEPT_LANGUAGE` and matches against each language's locale field:

```

1 if (isset($request->server['HTTP_ACCEPT_LANGUAGE'])) {
2     $browser_languages = explode(',',
        $request->server['HTTP_ACCEPT_LANGUAGE']);
3     foreach ($browser_languages as $browser_language) {
4         foreach ($languages as $key => $value) {
5             $locale = explode(',', $value['locale']);
6             if (in_array($browser_language, $locale)) {
7                 $detect = $key;
8             }
9         }
10    }
11 }

```

Listing 5: Browser language detection.

5.2 Admin Language Detection

The admin uses a simpler 1-level detection: the `config_admin_language` setting (stored in `setting` under `group='config', key='config_admin_language'`). No browser detection, no URL override.

```
1 $config->set('config_language_id',  
    $languages[$config->get('config_admin_language')]['language_id']);  
2 $language = new  
    Language($languages[$config->get('config_admin_language')]['directory']);  
3 $language->load($languages[$config->get('config_admin_language')]['filename']);
```

Listing 6: Admin language detection in `web/admin/index.php`.

5.3 language_id Propagation

Once detected, `config_language_id` is set in the `Config` singleton. Every storefront model that needs language-scoped data reads it via `$this->config->get('config_language_id')`:

```
1 $sql .= " LEFT JOIN " . DB_PREFIX . "description pd ON  
2     (pd.object_id = p.product_id  
3     AND pd.object_type = 'product'  
4     AND pd.language_id = '" . (int)$this->config->get('config_language_id')  
     . "') ";
```

Listing 7: `language_id` usage in the storefront product model.

6 Currency Detection

Currency detection is simpler than language detection:

```
1 // ?cc= GET parameter overrides config_currency  
2 if (isset($_GET['cc']) && $_GET['cc']) {  
3     $config->set('config_currency', $_GET['cc']);  
4 }
```

Listing 8: Currency detection in `app/shop/map.php`.

The `?cc= GET` parameter overrides the `config_currency` setting. There is no session or cookie persistence for currency — the override is per-request only. The `Currency` library singleton then uses `config_currency` as the active currency code and `config_currency_id` as the active currency ID.

7 The Polymorphic Object Spine

v1 identified the polymorphic object spine as the first of four cross-cutting patterns. v5 verifies and traces the implementation.

7.1 The Five Polymorphic Tables

All five tables use the same (`object_id`, `object_type`) pair as a composite soft foreign key. The `object_type` discriminator is a string like `'product'`, `'category'`, `'manufacturer'`, `'post'`, `'page'`, `'banner'`, `'download'`, `'post_category'`.

Table 2: The five polymorphic tables and their discriminator columns.

Table	Discriminator	Purpose
object_to_store	(object_id, object_type, store_id)	Assigns an object to a store.
object_to_category	(object_id, object_type, category_id)	Assigns an object to a category.
description	(object_id, object_type, language_id)	Localised rich-text content.
property	(object_id, object_type, group, key)	Free-form EAV key-value metadata.
url_alias	(object_id, object_type, language_id)	SEO URL slugs.

7.2 The object_to_store Writes — Inconsistent

object_to_store writes are inconsistent

The admin store model's `saveContent($store_id, $data)` method writes to `object_to_store` with different column patterns depending on the object type:

Legacy column names (NOT polymorphic):

- Products: `INSERT INTO object_to_store SET store_id=..., product_id=...`
- Categories: `INSERT INTO object_to_store SET store_id=..., category_id=...`
- Manufacturers: `INSERT INTO object_to_store SET store_id=..., manufacturer_id=...`

Polymorphic column names (correct):

- Downloads: `INSERT INTO object_to_store SET store_id=..., object_type='download', object_id=...`
- Pages: `INSERT INTO object_to_store SET store_id=..., object_type='page', object_id=...`
- Banners: `INSERT INTO object_to_store SET store_id=..., object_type='banner', object_id=...`

Missing store_id (bug):

- Posts: `INSERT INTO object_to_store SET object_type='post', object_id=...` — `store_id` is omitted!
- PostCategories: `INSERT INTO object_to_store SET object_type='post_category', object_id=...` — `store_id` is omitted!

The storefront product model reads `object_to_store` using the polymorphic pattern (`p2s.object_id = p.product_id AND p2s.object_type = 'product'`), but the admin writes using the legacy column name (`product_id`). The `object_to_store` table schema has both `object_id` and `product_id` columns (the latter is a legacy column not in the SQL dump but referenced in the code). This means the admin and storefront are using different columns of the same table.

7.3 The object_to_store Reads — Polymorphic

The storefront product model reads `object_to_store` using the polymorphic pattern:

```

1 $sql .= "LEFT JOIN " . DB_PREFIX . "object_to_store p2s ON
2   (p.product_id = p2s.object_id AND p2s.object_type = 'product') ";
3 $sql .= "LEFT JOIN " . DB_PREFIX . "store s ON
4   (s.store_id = p2s.store_id) ";

```

```

5 // ...
6 $criteria[] = " p2s.store_id = '" . (int)STORE_ID . "' ";

```

Listing 9: Storefront product model: polymorphic object_to_store read.

But some queries in the same model use the legacy `product_to_store` table (the OpenCart-specific junction):

```

1 LEFT JOIN " . DB_PREFIX . "product_to_store p2s ON
2     (p.product_id = p2s.product_id)
3     AND p2s.store_id = '" . (int) STORE_ID . "'

```

Listing 10: Storefront product model: legacy `product_to_store` read (coexists).

Both tables coexist. The `product_to_store` table is the OpenCart-specific junction; `object_to_store` is the polymorphic generalisation. Different queries hit different tables.

7.4 The description Table — Polymorphic + Language-Keyed

The `description` table stores localised rich-text content for any object:

```

1 $sql .= " LEFT JOIN " . DB_PREFIX . "description pd ON
2     (pd.object_id = p.product_id
3     AND pd.object_type = 'product'
4     AND pd.language_id = '" . (int)$this->config->get('config_language_id')
5     . "') ";

```

Listing 11: Storefront product model: description join.

The composite unique key (`object_id`, `object_type`, `language_id`) ensures one description per object per language. The table has columns for `title`, `description` (rich HTML), `seo_title`, `meta_description`, `meta_keywords`, and `params` (a serialised PHP blob).

7.5 The property EAV Table — Polymorphic + Grouped

The `property` table stores free-form key-value metadata for any object, grouped by a `group` string:

Table 3: The property table structure.

Column	Type	Purpose
<code>property_id</code>	INT	Primary key.
<code>store_id</code>	INT	The store (nullable; NULL = all stores).
<code>object_id</code>	INT	The object's primary key.
<code>object_type</code>	VARCHAR(100)	The discriminator (e.g. 'product', 'campaign').
<code>group</code>	VARCHAR(100)	The property group (e.g. 'product_attribute', 'mail_server').
<code>key</code>	VARCHAR(100)	The property key.
<code>value</code>	TEXT	The property value.

The composite unique key (`object_id`, `object_type`, `group`, `key`) ensures one value per key per group per object. The `store_id` column allows per-store property overrides (though it is nullable, meaning a property can apply to all stores).

7.6 The url_alias Table — Polymorphic + Language-Keyed

Two unique keys enforce correctness: `UNIQUE (keyword, language_id)` (a slug is unique within a language) and `UNIQUE (object_id, language_id, object_type)` (each object has at most one slug per language per type). The SEO URL rewriter (`ControllerCommonSeoUrl`) queries this table with `WHERE keyword = '...' AND language_id = config_language_id`.

Table 4: The `url_alias` table structure.

Column	Type	Purpose
<code>url_alias_id</code>	INT	Primary key.
<code>object_id</code>	INT	The object's primary key.
<code>language_id</code>	INT	The language.
<code>object_type</code>	VARCHAR(50)	The discriminator.
<code>query</code>	VARCHAR(255)	The internal query string (e.g. <code>product_id=42</code>).
<code>keyword</code>	VARCHAR(255)	The SEO slug (e.g. <code>my-first-product</code>).

8 Storefront Filtering by `store_id` and `language_id`

Every storefront catalog model filters by both `store_id` and `language_id`. The pattern is consistent across products, categories, manufacturers, and reviews.

8.1 The Product Model

```

1  $sql = "SELECT *, pd.title AS name, p.product_id
2  FROM " . DB_PREFIX . "product p
3  LEFT JOIN " . DB_PREFIX . "description pd ON
4  (pd.object_id = p.product_id
5  AND pd.object_type = 'product'
6  AND pd.language_id = '" .
7  (int)$this->config->get('config_language_id') . "')
8  LEFT JOIN " . DB_PREFIX . "object_to_store p2s ON
9  (p.product_id = p2s.object_id AND p2s.object_type = 'product')
10 LEFT JOIN " . DB_PREFIX . "store s ON
11 (s.store_id = p2s.store_id)";
12
13 // Store filter
14 if (!empty($data['store_id'])) {
15     if (is_array($data['store_id'])) {
16         $criteria[] = " p2s.store_id IN ('" . implode("'", $data['store_id']) . "') ";
17     } else {
18         $criteria[] = " p2s.store_id = '" . intval($data['store_id']) . "' ";
19     }
20 } else {
21     $criteria[] = " p2s.store_id = '" . (int)STORE_ID . "' ";
22 }

```

Listing 12: Storefront product model: the store + language filter pattern.

The `LEFT JOIN` description `pd` with the `language_id` filter ensures only the current language's description is returned. The `LEFT JOIN` `object_to_store` `p2s` with the `store_id` filter ensures only products assigned to the current store are returned.

8.2 The Category Model

Same pattern, with `object_type = 'category'`:

```

1  $sql .= " LEFT JOIN " . DB_PREFIX . "object_to_store c2s ON
2  (p2c.category_id = c2s.object_id AND c2s.object_type = 'category') ";
3  $criteria[] = " c2s.store_id = '" . (int)STORE_ID . "' ";

```

Listing 13: Storefront category model: store filter.

8.3 The Manufacturer Model

Same pattern, with `object_type = 'manufacturer'`:

```
1 $sql .= " LEFT JOIN " . DB_PREFIX . "object_to_store m2s ON
2     (m2s.object_id = m.manufacturer_id AND m2s.object_type = 'manufacturer')
3     ";
4 $criteria[] = " m2s.store_id = '" . (int)STORE_ID . "' ";
```

Listing 14: Storefront manufacturer model: store filter.

9 Widget Composition by `object_type` and `object_id`

v3 covered the widget rendering pipeline. v5 adds the per-page widget composition mechanism: how the session's `object_type` and `object_id` override the widget tree.

9.1 The Session `object_type` / `object_id` Protocol

When a product page loads, the product controller sets the session:

```
1 // app/shop/controller/store/product.php
2 $this->session->clear('object_type');
3 $this->session->clear('object_id');
4 $this->session->clear('landing_page');
5 $this->session->set('object_type', 'product');
6 $this->session->set('object_id', $product_id);
```

Listing 15: Product page sets session `object_type`.

When a category page loads:

```
1 // app/shop/controller/store/category.php
2 $this->session->clear('object_type');
3 $this->session->clear('object_id');
4 $this->session->clear('landing_page');
5 $this->session->set('object_type', 'category');
6 $this->session->set('object_id', $this->data['category_id']);
```

Listing 16: Category page sets session `object_type`.

When the home page loads, the session is cleared:

```
1 // app/shop/controller/common/home.php
2 $this->session->set('landing_page', 'common/home');
3 // (object_type and object_id are NOT set - they retain whatever the
4 // previous page set, or are clear if this is the first page)
```

Listing 17: Home page clears session `object_type`.

9.2 `Controller::loadWidgets()` Reads the Session

```
1 // After loading the default widget tree...
2 if ($session->has('object_type') || $session->has('object_id')) {
3     // loading widgets just for this object_type
4     if ($session->has('object_type'))
5         $widgets->object_type = $session->get('object_type');
6     if ($session->has('object_id'))
7         $widgets->object_id = $session->get('object_id');
8
9     // Rebuild params with object_type/object_id filter
```

```

10     $params['object_type'] = $session->get('object_type');
11     $params['object_id']   = $session->get('object_id');
12
13     // Query NecoWidget again with the object filter
14     $rows = $widgets->getRows($params, !$this->user->getId());
15     // ... merge the per-object widget rows into $this->children
16 }

```

Listing 18: Controller::loadWidgets() reads session object_type for per-object widget overrides.

So loadWidgets() runs *two* queries: first the default widget tree (without object_type/object_id filter), then a second query with the filter. The per-object widget rows are *merged* into the default tree, so a product page sees both the default widgets (e.g. header, footer, navigation) and the product-specific widgets (e.g. “related products”, “product attributes”).

9.3 NecoWidget::getRows() Filters by object_type

NecoWidget::getRows() adds LIKE predicates when object_type and object_id are set:

```

1  if (isset($data['object_type']) && !empty($data['object_type'])) {
2      $criteria[] = " p.'value' LIKE '%object_type=" .
3          $db->escape($data['object_type']) . "%' ";
4  } else {
5      $criteria[] = " p.'value' NOT LIKE '%object_type%' ";
6  }
7
8  if (isset($data['object_id']) && !empty($data['object_id'])) {
9      $criteria[] = " p.'value' LIKE '%object_id=" .
10         $db->escape($data['object_id']) . "%' ";
11 } else {
12     $criteria[] = " p.'value' NOT LIKE '%object_id%' ";
13 }

```

Listing 19: NecoWidget::getRows() adds object_type/object_id filters.

This is the same LIKE '%key=value%' pattern v3 documented for device filtering. The widget’s serialised settings blob contains object_type=product and object_id=42 strings, and the query matches them with LIKE.

9.4 The widget_landing_page Table

The widget_landing_page table binds a widget to a specific landing page (route) and object type:

Table 5: The widget_landing_page table structure.

Column	Type	Purpose
widget_landing_page_id	INT	Primary key.
widget_id	INT	The widget.
landing_page	VARCHAR(150)	The route (e.g. store/product, common/home, all).
object_type	VARCHAR(255)	The object type (e.g. product, category).

The admin widget model queries this table to filter widgets by landing page:

```

1  $criteria[] = " w.'widget_id' IN (
2      SELECT widget_id FROM " . DB_PREFIX . "widget_landing_page
3      WHERE landing_page = ' " . $this->db->escape($this->landing_page) . "'
4          OR landing_page = 'all'
5  ) ";

```

Listing 20: Admin widget model: `widget_landing_page` filter.

So a widget is visible on a specific route (e.g. `store/product`) or on all routes (`all`). The `object_type` column further refines the visibility (e.g. a widget visible on `store/product` with `object_type = 'product'`).

10 The $S \times L$ Composition Matrix

The combination of `store_id` and `language_id` forms a $S \times L$ matrix of content surfaces. Each cell in the matrix is a unique (store, language) pair with its own:

- Settings (from the `setting` table, scoped by `store_id`).
- Products, categories, manufacturers (from `object_to_store`, scoped by `store_id`).
- Descriptions, SEO titles, meta tags (from the `description` table, scoped by `object_id` + `object_type` + `language_id`).
- SEO URL slugs (from `url_alias`, scoped by `object_id` + `object_type` + `language_id`).
- Widget instances (from the `property` EAV table, scoped by `store_id` in the widget row's settings).
- Theme (from `config_template`, scoped by `store_id`).
- Currency (from `config_currency`, scoped by `store_id`, overrideable by `?cc= GET`).

	English	Spanish	Portuguese
Store 0	Global EN	Global ES	Global PT
Store 1	Shop A EN	Shop A ES	Shop A PT
Store 9	Mobile EN	Mobile ES	Mobile PT

Figure 2: The $S \times L$ composition matrix. Each cell is a unique (store, language) pair with its own settings, products, descriptions, SEO slugs, and widgets. Store 0 is the global/default store; Store 1 is a custom storefront; Store 9 is the mobile entry point.

10.1 How the Matrix is Realised

The matrix is not a single table; it is the composition of five filtering dimensions at query time:

1. `store_id` filters the `setting` table (settings).
2. `store_id` filters `object_to_store` (which products/categories/manufacturers are visible).
3. `language_id` filters `description` (which title/body/SEO text is returned).
4. `language_id` filters `url_alias` (which SEO slug is used).
5. `store_id` + `landing_page` + `object_type` + `object_id` filter the `property` EAV table (which widgets are shown).

There is no single “content” table that is keyed on (`store_id`, `language_id`). Instead, the storefront models compose the five filters at query time, producing a unique result set for each (store, language) pair.

11 Multi-Store Request Lifecycle Trace

This trace walks through a request to `https://shop-a.example.com/products/my-product` where `shop-a` is a custom store (`store_id=1`), the active language is Spanish, and the product is `product_id=42`.

11.1 Store Detection

1. `web/index.php` receives the request.
2. Strategy 1 (URL path folder match): splits `REQUEST_URI` by `/`. Checks `products` — no store folder match. Checks `my-product` — no store folder match.
3. Strategy 2 (`?store_id`): not set.
4. Strategy 3 (subdomain regex): a `preg_match` call on the hostname with a pattern that captures the leftmost label. But the domain is `example.com`, not `necoyoad.com`. No match.

Wait — the subdomain regex is hard-coded to `necoyoad.com`. For `example.com`, none of the three strategies match, so `$matches[1]` is not set, and the system falls back to `app/shop/config.php` with `STORE_ID = 0`.

For this trace, we assume the domain is `shop-a.necoyoad.com`, so strategy 3 matches and `$matches[1] = 'shop-a'`.

5. `file_exists('app/shop-a/config.php')` — yes (the admin created a custom store folder).
6. `require_once('app/shop-a/config.php')` — sets `STORE_ID = 1` and `DIR_APPLICATION` to `app/shop-a/`.

11.2 Settings Load

7. `system/startup.php` loads the engine, libraries, and Hooks/Events system.
8. `app/shop-a/map.php` (or `app/shop/map.php` if the custom store reuses the shop code) loads settings: `SELECT * FROM setting WHERE store_id = 1`. Returns all settings for store 1 (including `config_template = 'shop-a-theme'`, `config_language = 'es-ve'`, `config_currency = 'VES'`, etc.).
9. `$config->set($key, $value)` for each row.
10. `$config->set('config_store_id', 1)`.

11.3 Language Detection

10. `SELECT * FROM language` — returns all active languages.
11. Language detection: no `?language=` or `?hl=` GET param. Session has `language = 'es-ve'` (from a previous request). `$code = 'es-ve'`.
12. `$config->set('config_language_id', 2)` (Spanish is `language_id=2`).
13. `$config->set('config_language', 'es-ve')`.
14. Load the Spanish language file: `new Language('spanish');`
`$language->load('spanish');`

11.4 SEO URL Resolution

14. The `common/seo_url` pre-action runs.
15. `_route_ = 'products/my-product'`.
16. Split by `/`: `['products', 'my-product']`.
17. Segment `'products'`: `SELECT * FROM store WHERE folder = 'products'` — no match. Strip.
18. Segment `'my-product'`: `SELECT * FROM url_alias WHERE keyword = 'my-product' AND language_id = 2`. Returns `query = 'product_id=42', object_type = 'product', object_id = 42`.
19. `$request->get['product_id'] = 42`.
20. Route resolution: `$request->get['r'] = 'store/product'`.

11.5 Product Controller

20. `ControllerStoreProduct::index()` runs.
21. Sets session: `object_type = 'product', object_id = 42, landing_page = 'store/product/index'`.
22. Loads the product: `SELECT *, pd.title AS name, p.product_id FROM product p LEFT JOIN description pd ON (pd.object_id = p.product_id AND pd.object_type = 'product' AND pd.language_id = 2) LEFT JOIN object_to_store p2s ON (p.product_id = p2s.object_id AND p2s.object_type = 'product') LEFT JOIN store s ON (s.store_id = p2s.store_id) WHERE p.product_id = 42 AND p2s.store_id = 1`.
23. The product is returned (it exists in store 1 with a Spanish description).
24. `loadWidgets('content_top'), loadWidgets('content_bottom'), loadWidgets('column_left'), loadWidgets('column_right')`.
25. Each `loadWidgets()` call:
 - (a) Queries the default widget tree: `SELECT * FROM property WHERE object_type='widget_rows' AND p.value LIKE '%store_id=1%' AND p.value LIKE '%landing_page=store/product/index%' AND p.value LIKE '%show_in_desktop=on%'`.
 - (b) Queries the per-object widget tree: `... AND p.value LIKE '%object_type=product%' AND p.value LIKE '%object_id=42%'`.
 - (c) Merges the per-object widgets into the default tree.
26. Adds layout children: `common/header, common/footer, common/column_left, common/column_right`.
27. Sets template to `shop-a-theme/store/product.tpl` (or `choroni/store/product.tpl` if the custom theme is missing the file).
28. `render(true)` — renders the page with the store-scoped, language-scoped, object-scoped widget tree.

12 Cross-Cutting Findings

12.1 No Default + Override Settings Merge

Each store is an settings island

The storefront loads `SELECT * FROM setting WHERE store_id = STORE_ID` — only the current store. There is no merge with `store_id=0`. Each store must have its own complete set of settings. The admin clones `store_id=0` settings when creating a new store, but subsequent changes to `store_id=0` do not propagate.

12.2 Inconsistent `object_to_store` Writes

Legacy and polymorphic column names coexist

The admin store model writes products, categories, and manufacturers using legacy column names (`product_id`, `category_id`, `manufacturer_id`), but writes downloads, pages, posts, and banners using the polymorphic pair (`object_id`, `object_type`). The storefront reads using the polymorphic pair. The `object_to_store` table has both legacy and polymorphic columns; the admin and storefront are using different columns of the same table.

12.3 Missing `store_id` in Posts Insert

Posts and post-categories `INSERT` omit `store_id`

The admin store model's `saveContent()` method inserts posts and post-categories into `object_to_store` *without* setting the `store_id` column. This means posts and post-categories are not actually scoped to any store — they are either global (if `store_id` defaults to 0) or unpredictable (if the column has no default).

12.4 Dual Table Usage

`object_to_store` and `product_to_store` coexist

The storefront product model uses both `object_to_store` (polymorphic) and `product_to_store` (legacy OpenCart) in different queries. Both tables coexist in the schema. Some queries hit one, some the other. This means product-to-store assignments must be maintained in both tables for the product to appear consistently.

12.5 The Subdomain Regex is Hard-Coded

Subdomain detection only works for `necoyoad.com`

The subdomain regex (a `preg_match` call on `$_SERVER['SERVER_NAME']` with a pattern targeting `necoyoad.com`) is hard-coded to `necoyoad.com`. For any other domain (e.g. `shop-a.example.com`), the subdomain strategy fails, and the system falls back to `app/shop/config.php` with `STORE_ID = 0`.

12.6 Language Detection Has No URL-Path Strategy

The language detection chain checks GET params, session, cookie, and browser — but not the URL path. So `example.com/en/products` and `example.com/es/productos` would both use the same language (whatever the session/cookie/browser determines). Language switching is done via the `?language=` or `?hl=` GET parameter (typically from a dropdown in the header), not via URL path prefixing.

12.7 Currency Override is Per-Request Only

The `?cc=` GET parameter overrides `config_currency` for the current request only. There is no session or cookie persistence. So every link on the page must include `?cc=<code>` to maintain the currency across navigation — otherwise the next request reverts to the store’s default currency.

12.8 Widget Composition is Two Queries, Not One

`Controller::loadWidgets()` runs *two* `NecoWidget::getRows()` queries when the session has `object_type` set: one for the default widget tree (without the object filter), one for the per-object widget tree (with the filter). The per-object widgets are merged into the default tree. This means a product page with 5 layout positions fires $5 \times 2 = 10$ `NecoWidget` queries (plus the `getCols()` sub-queries within each).

13 Closing Observations

13.1 The Multi-Store Architecture is a Data-Level Multi-Tenancy

Necoyoad’s multi-store architecture is a data-level multi-tenancy: all stores share the same codebase and the same database, but each store sees only its own data (filtered by `store_id` at query time). There is no separate database per store, no separate codebase per store, and no separate process per store. The `STORE_ID` constant, set at boot by the entry point, is the sole tenant discriminator.

13.2 The Polymorphic Object Spine is the Architectural Glue

The five polymorphic tables (`object_to_store`, `object_to_category`, `description`, `property`, `url_alias`) are the architectural glue that ties content to stores, languages, categories, and SEO URLs. Without them, every content type (product, category, manufacturer, post, page, banner, download) would need its own set of junction and localisation tables. The polymorphic pattern lets the platform add new content types without schema changes — a new type just needs to populate the `object_type` discriminator.

13.3 The Inconsistencies are Evolutionary Scars

The inconsistent `object_to_store` writes (legacy vs. polymorphic column names), the dual table usage (`object_to_store` vs. `product_to_store`), the missing `store_id` in the posts insert, and the hard-coded subdomain regex are all evolutionary scars. The platform started as an OpenCart fork (with per-type junction tables like `product_to_store`), evolved a polymorphic generalisation (`object_to_store`), and never fully migrated. The result is a codebase where both patterns coexist, sometimes in the same file.

13.4 The Widget Composition Mechanism is the Most Sophisticated Part

The widget composition by `object_type` and `object_id` is the most sophisticated part of the multi-store architecture. It allows the admin to configure a widget (e.g. “related products”) to

appear only on product pages, or only on a specific product, or only on a specific category — all via the serialised settings blob and the LIKE `'%key=value%'` query pattern. The two-query merge (default tree + per-object tree) ensures that per-object widgets are additive, not replacement.

13.5 Final Note

v5 confirms that the Necoyoad multi-store/multi-language architecture is a data-level multi-tenancy with a polymorphic object spine. The $S \times L$ composition matrix is realised at query time by five filtering dimensions (`store_id`, `language_id`, `object_type`, `object_id`, `landing_page`), not by a single schema-level construct. The inconsistencies (legacy vs. polymorphic columns, dual table usage, missing `store_id` in posts) are evolutionary scars from the OpenCart-to-Necoyoad migration, documented here descriptively, not prescriptively.

A Glossary

Table 6: Glossary of multi-store/multi-language terms.

Term	Meaning
<code>STORE_ID</code>	A global PHP constant set at boot by the per-app config file. The sole tenant discriminator.
$S \times L$ matrix	The composition of <code>store_id</code> and <code>language_id</code> , forming a unique content surface per (store, language) pair.
Polymorphic object spine	The five tables (<code>object_to_store</code> , <code>object_to_category</code> , <code>description</code> , <code>property</code> , <code>url_alias</code>) that use the (<code>object_id</code> , <code>object_type</code>) pair as a composite soft FK.
<code>object_type</code>	The string discriminator (e.g. <code>'product'</code> , <code>'category'</code>) that identifies the kind of object in a polymorphic association.
Settings island	The finding that each store loads only its own <code>store_id</code> settings, with no merge with <code>store_id=0</code> .
Legacy column	A non-polymorphic column name (e.g. <code>product_id</code>) used in <code>object_to_store</code> writes for products, categories, and manufacturers. Coexists with the polymorphic <code>object_id</code> column.
Widget composition	The mechanism by which the session's <code>object_type</code> and <code>object_id</code> override the widget tree on a per-page basis. Two queries: default tree + per-object tree, merged.
<code>landing_page</code>	The route stored in the session (e.g. <code>store/product/index</code>), used by <code>NecoWidget</code> to filter widgets by page.